

ЗВІТ

до лабораторної роботи № 6

на тему:

**"Поглиблене вивчення MongoDB: оптимізація продуктивності,
використання шардінгу та реплікації."**

Виконала:

студентка групи МІТ-31

Йовхимиш Діана

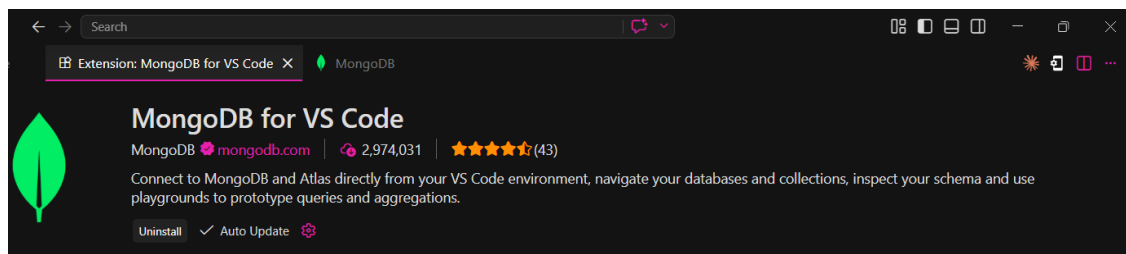
Мета роботи: ознайомитися з методами підвищення продуктивності MongoDB. Навчитися використовувати індекси для оптимізації запитів. Дослідити механізми реплікації та шардінгу.

Хід роботи

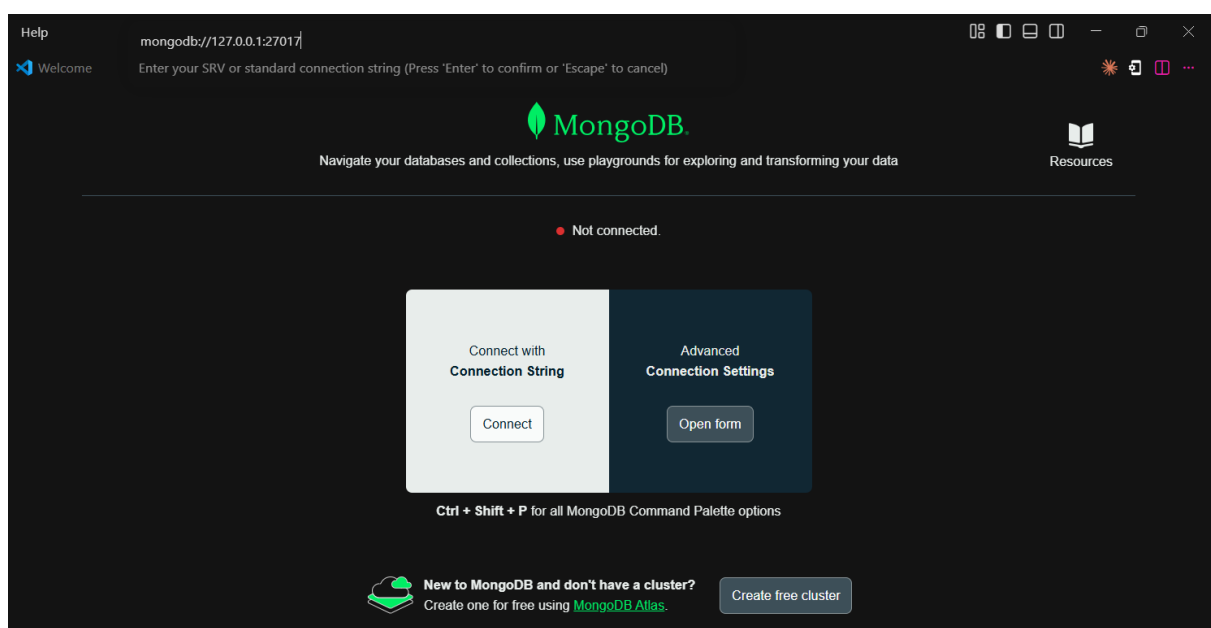
Частина 1: Оптимізація продуктивності запитів

1. Створення локальної бази MongoDB у VS Code

1. Встановіть розширення **MongoDB for VS Code**.

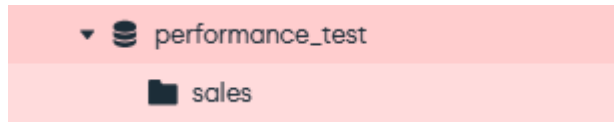


2. Відкрийте вкладку **MongoDB Explorer** у VS Code.
3. Натисніть **+ Add Connection**, введіть `mongodb://127.0.0.1:27017` та підключіться.



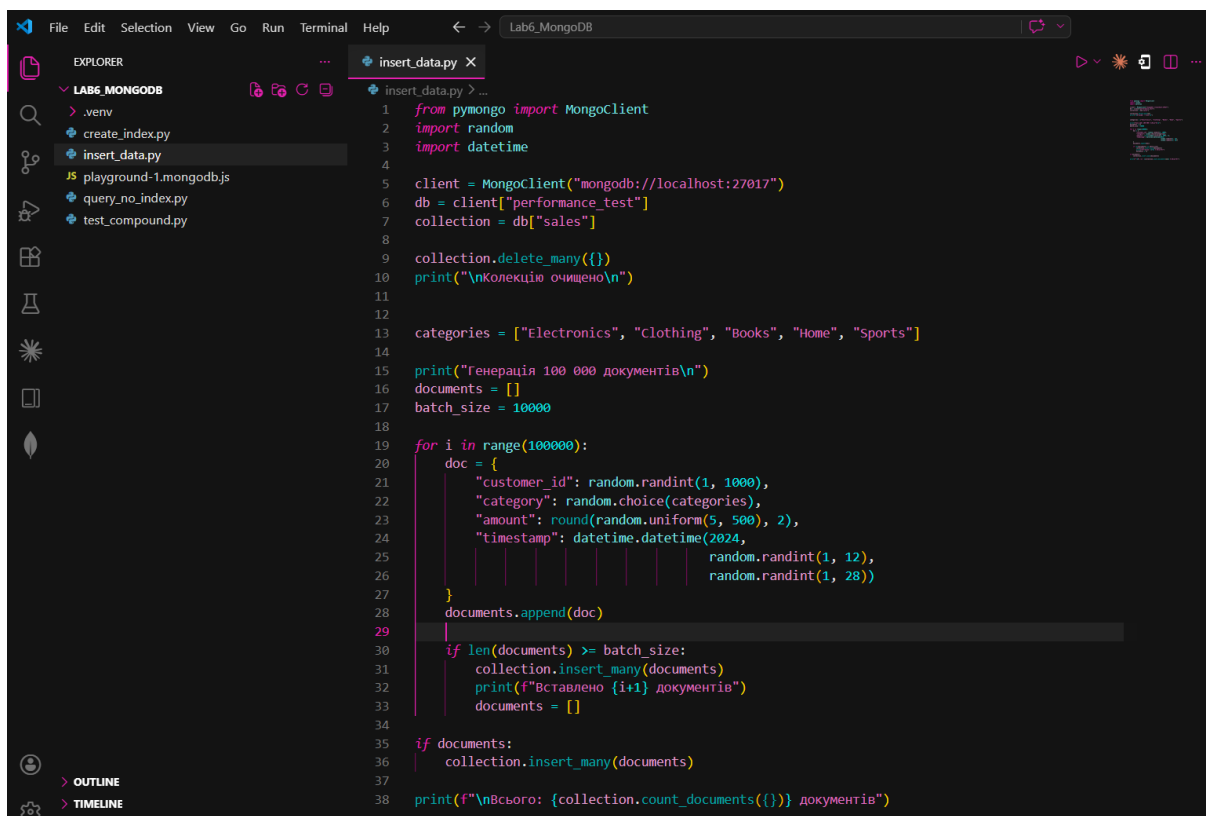
4. У вкладці MongoDB Explorer створіть нову базу performance_test.

5. Створіть колекцію sales.



2. Генерація та вставка тестових даних

1. Написати Python-скрипт **insert_data.py** для генерації 100 000 документів:



```
1 from pymongo import MongoClient
2 import random
3 import datetime
4
5 client = MongoClient("mongodb://localhost:27017")
6 db = client["performance_test"]
7 collection = db["sales"]
8
9 collection.delete_many({})
10 print("\nКолекцію очищено\n")
11
12 categories = ["Electronics", "Clothing", "Books", "Home", "Sports"]
13
14 print("Генерація 100 000 документів\n")
15 documents = []
16 batch_size = 10000
17
18 for i in range(100000):
19     doc = {
20         "customer_id": random.randint(1, 1000),
21         "category": random.choice(categories),
22         "amount": round(random.uniform(5, 500), 2),
23         "timestamp": datetime.datetime(2024,
24                                     random.randint(1, 12),
25                                     random.randint(1, 28))
26     }
27     documents.append(doc)
28
29 if len(documents) >= batch_size:
30     collection.insert_many(documents)
31     print(f"Вставлено {i+1} документів")
32     documents = []
33
34 if documents:
35     collection.insert_many(documents)
36
37 print(f"\nВсього: {collection.count_documents({})} документів")
```

2. Виконати скрипт та переконатися, що дані вставлено:

```
> ▾ TERMINAL
Admin@DESKTOP-2RGOT4B MINGW64 ~/Desktop/Lab6_MongoDB
$ python insert_data.py

Колекцію очищено

Генерація 100 000 документів

Вставлено 10000 документів
Вставлено 20000 документів
Вставлено 30000 документів
Вставлено 40000 документів
Вставлено 50000 документів
Вставлено 60000 документів
Вставлено 70000 документів
Вставлено 80000 документів
Вставлено 90000 документів
Вставлено 100000 документів

Всього: 100000 документів
```

Виконала скрипт **insert_data.py**. Колекцію попередньо очищено, після чого згенеровано та вставлено 100 000 документів батчами по 10 000. Термінал виводив прогрес вставки після кожного батчу. Результат: всього вставлено **100 000 документів** у колекцію **sales**.

3. Виконати пошуковий запит без індексу та виміряти час виконання (query_no_index.py).

Створила скрипт **query_no_index.py** який виконує пошук документів з категорії Electronics та вимірює час виконання. На цьому етапі індекси відсутні — MongoDB виконує повний перебір усіх 100 000 документів колекції:

```
File Edit Selection View Go Run Terminal Help
Lab6_MongoDB
EXPLORER
LAB6_MONGODB
  .venv
  create_index.py
  insert_data.py
  playground-1.mongodb.js
  query_no_index.py
  test_compound.py
query_no_index.py
1 from pymongo import MongoClient
2 import time
3
4 client = MongoClient("mongodb://localhost:27017")
5 db = client["performance_test"]
6 collection = db["sales"]
7
8 start_time = time.time()
9 results = list(collection.find({"category": "Electronics"}))
10 end_time = time.time()
11
12 print(f"Знайдено документів: {len(results)}")
13 print(f"Час виконання: {end_time - start_time:.6f} секунд")
```

```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE

>  ✓ TERMINAL

Admin@DESKTOP-2RGOT4B MINGW64 ~/Desktop/Lab6_MongoDB
$ python query_no_index.py
Час виконання: 0.262048 секунд
```

4. Створити індекс на полі category (create_index.py).

Створила Python-скрипт **create_index.py** для створення індексу на полі category колекції sales.

Скрипт спочатку видаляє всі існуючі індекси (крім _id), потім створює новий індекс та виводить список поточних індексів:

```
File Edit Selection View Go Run Terminal Help  Lab6_MongoDB

EXPLORER
  LAB6_MONGODB
    .venv
    create_index.py
    insert_data.py
    playground-1.mongodb.js
    query_no_index.py
    test_compound.py

create_index.py > ...
1  from pymongo import MongoClient
2
3  client = MongoClient("mongodb://localhost:27017")
4  db = client["performance_test"]
5  collection = db["sales"]
6
7  for index in collection.list_indexes():
8      if index['name'] != '_id':
9          collection.drop_index(index['name'])
10
11  collection.create_index("category")
12  print("Індекс на полі category створено!")
13
14  print("\nПоточні індекси:")
15  for index in collection.list_indexes():
16      print(f"  - {index['name']}: {index['key']}")
```

```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE

>  ✓ TERMINAL

Admin@DESKTOP-2RGOT4B MINGW64 ~/Desktop/Lab6_MongoDB
$ python create_index.py
Поточні індекси:
  - _id: SON([('_id', 1)])
  - category_1: SON([('category', 1)])
```

Після виконання скрипту колекція містить два індекси: **_id** та **category_1**.

5. Повторно виконати запит та порівняти час виконання:

```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE

>  ▾ TERMINAL

Admin@DESKTOP-2RGOT4B MINGW64 ~/Desktop/Lab6_MongoDB
• $ python query_no_index.py
Знайдено документів: 19830
Час виконання: 0.077528 секунд
```

Після створення індексу на полі `category` повторно виконала скрипт `query_no_index.py`. Час виконання запиту скоротився з **0.262048 секунд** до **0.077528 секунд** — прискорення у ~ 3.4 рази. Це підтверджує що індекс значно підвищує продуктивність пошукових запитів.

6. Дослідити складений індекс (`category, timestamp`) та його вплив на продуктивність.

Створила скрипт `test_compound.py` який порівнює три варіанти виконання запиту по полях `category` та `timestamp`. Скрипт послідовно виконує запит без індексу, з простим індексом і зі складеним індексом, вимірюючи час кожного варіанту:

```
File Edit Selection View Go Run Terminal Help  < -> Lab6_MongoDB

EXPLORER
  LAB6_MONGODB
    .venv
    create_index.py
    insert_data.py
    playground-1.mongodb.js
    query_no_index.py
    test_compound.py

test_compound.py > ...
1  from pymongo import MongoClient, ASCENDING
2  import time
3  from datetime import datetime
4
5  client = MongoClient("mongodb://localhost:27017")
6  db = client["performance_test"]
7  collection = db["sales"]
8
9
10 print("Дослідження складеного індексу (category, timestamp)")
11
12 print("\n1. Запит без індексу")
13
14
15 collection.drop_indexes()
16 start_time = time.time()
17
18
19 results = list(collection.find({
20     "category": "Electronics",
21     "timestamp": {"$gte": datetime(2024, 1, 1), "$lte": datetime(2024, 6, 30)}
22 }))
23
24 end_time = time.time()
25 time_no_index = end_time - start_time
26
27 print(f"Знайдено документів: {len(results)}")
28 print(f"Час виконання: {time_no_index:.6f} секунд")
29
30
31 print("\n2. Запит з простим індексом (тільки поле category)")
32
33 collection.create_index("category")
34
35 start_time = time.time()
36
```

```
File Edit Selection View Go Run Terminal Help Lab6_MongoDB
EXPLORER
LAB6_MONGODB
  .venv
  create_index.py
  insert_data.py
  playground-1.mongodb.js
  query_no_index.py
  test_compound.py
Devices
OUTLINE
TIMELINE

test_compound.py > ...
35 start_time = time.time()
36
37 results = list(collection.find({
38     "category": "Electronics",
39     "timestamp": {"$gte": datetime(2024, 1, 1), "$lte": datetime(2024, 6, 30)}
40 })))
41
42 end_time = time.time()
43 time_simple_index = end_time - start_time
44
45 print(f"Знайдено документів: {len(results)}")
46 print(f"Час виконання: {time_simple_index:.6f} секунд")
47
48
49 print("\n3. Запит зі складеним індексом (category + timestamp)")
50
51 collection.drop_indexes()
52 collection.create_index([
53     ("category", ASCENDING),
54     ("timestamp", ASCENDING)
55 ])
56
57 start_time = time.time()
58
59 results = list(collection.find({
60     "category": "Electronics",
61     "timestamp": {"$gte": datetime(2024, 1, 1), "$lte": datetime(2024, 6, 30)}
62 })))
63
64 end_time = time.time()
65 time_compound_index = end_time - start_time
66
67 print(f"Знайдено документів: {len(results)}")
68 print(f"Час виконання: {time_compound_index:.6f} секунд")
69
70
71
72 print("\nПорівняння швидкості:")
```

```
File Edit Selection View Go Run Terminal Help Lab6_MongoDB
EXPLORER
LAB6_MONGODB
  .venv
  create_index.py
  insert_data.py
  playground-1.mongodb.js
  query_no_index.py
  test_compound.py
Devices
OUTLINE
TIMELINE

test_compound.py > ...
67 print(f"Знайдено документів: {len(results)}")
68 print(f"Час виконання: {time_compound_index:.6f} секунд")
69
70
71
72 print("\nПорівняння швидкості:")
73 speedup_simple = time_no_index / time_simple_index
74 if speedup_simple > 1:
75     print(f"Простий індекс прискорив запит у {speedup_simple:.2f} разів")
76 else:
77     slowdown = 1 / speedup_simple
78     print(f"Простий індекс не допоміг (сповільнення у {slowdown:.2f} разів)")
79
80
81 speedup_compound = time_no_index / time_compound_index
82 print(f"Складений індекс прискорив запит у {speedup_compound:.2f} разів")
83
84 speedup_vs_simple = time_simple_index / time_compound_index
85 print(f"Складений індекс швидший за простий у {speedup_vs_simple:.2f} разів")
86
87
88 print("\nТаблиця результатів:")
89
90 print(f"{'Тип індексу':<25} {'Час':<12} {'Прискорення':<12}")
91 print("-" * 50)
92 print(f"{'Без індексу':<25} {time_no_index:.4f} c {'1.00x':<12}")
93 print(f"{'Простий':<25} {time_simple_index:.4f} c {'speedup_simple:.2f}x")
94 print(f"{'Складений':<25} {time_compound_index:.4f} c {'speedup_compound:.2f}x")
95 print("-" * 50)
```

```
Admin@DESKTOP-2RGOT4B MINGW64 ~/Desktop/Lab6_MongoDB
● $ python test_compound.py
Дослідження складеного індексу (category, timestamp)

1. Запит без індексу
Знайдено документів: 9895
Час виконання: 0.172098 секунд

2. Запит з простим індексом (тільки поле category)
Знайдено документів: 9895
Час виконання: 0.182464 секунд

3. Запит зі складеним індексом (category + timestamp)
Знайдено документів: 9895
Час виконання: 0.074439 секунд

Порівняння швидкості:
Простий індекс не допоміг (сповільнення у 1.06 разів)
Складений індекс прискорив запит у 2.31 разів
Складений індекс швидший за простий у 2.45 разів

Таблиця результатів:
Тип індексу          Час          Прискорення
-----
Без індексу          0.1721 с     1.00x
Простий              0.1825 с     0.94x
Складений            0.0744 с     2.31x
-----
```

Простий індекс не дав переваги для запиту з двома умовами фільтрації (сповільнення 1.06х), тоді як складений індекс прискорив запит у **2.31 рази** і виявився у **2.45 рази швидшим** за простий індекс.

Частина 2: Налаштування реплікації у MongoDB

1. Запустити три екземпляри MongoDB у режимі реплікації.

Створила три окремі директорії для зберігання даних кожного вузла replica set:

```
Командная строка
Microsoft Windows [Version 10.0.26200.7840]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Admin>mkdir D:\data\rs0-1
C:\Users\Admin>mkdir D:\data\rs0-2
C:\Users\Admin>mkdir D:\data\rs0-3
C:\Users\Admin>
```


Відкрила три окремих вікна PowerShell і в кожному запущено окремий екземпляр MongoDB з параметром --replSet rs0:

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\Admin> mongod --replSet rs0 --port 27017 --dbpath D:\data\rs0-1 --bind_ip localhost
```

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

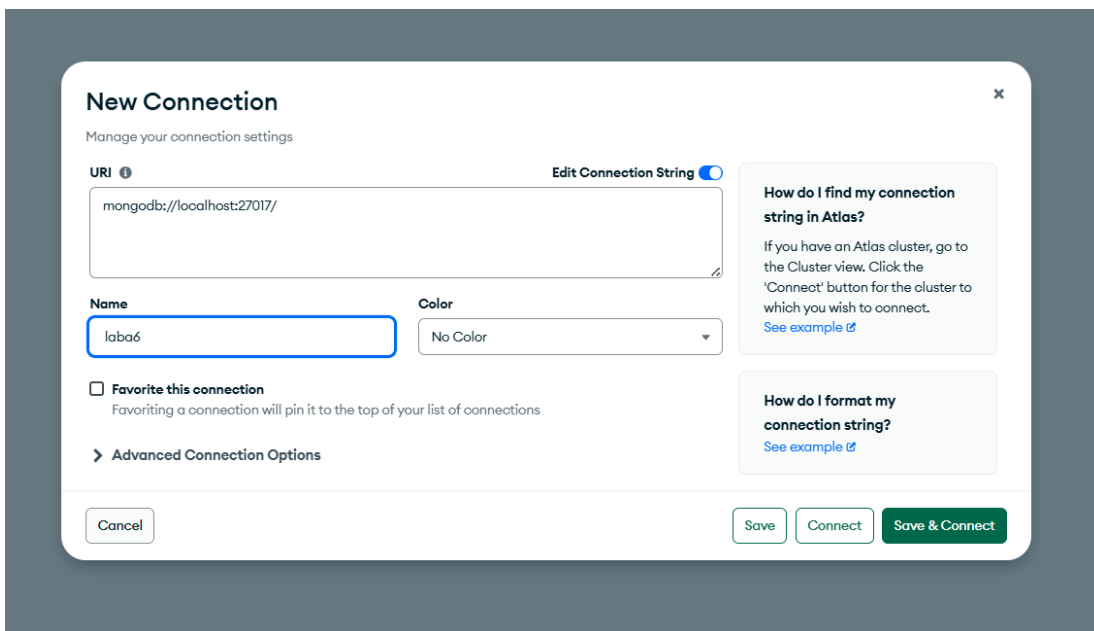
PS C:\Users\Admin> mongod --replSet rs0 --port 27018 --dbpath D:\data\rs0-2 --bind_ip localhost
```

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\Admin> mongod --replSet rs0 --port 27019 --dbpath D:\data\rs0-3 --bind_ip localhost
```

У MongoDB Compass створено нове підключення mongodb://localhost:27017 з назвою **laba6**:



Далі через вкладку Shell успішно виконала ініціалізацію replica set та додавання вузлів:

```
> rs.initiate()
< {
  info2: 'no configuration specified. Using a default configuration for the set',
  me: 'localhost:27017',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777390823, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777390823, i: 1 })
}
```

```
> rs.add("localhost:27018")
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777390865, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777390865, i: 1 })
}
```

```
> rs.add("localhost:27019")
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777390884, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777390884, i: 1 })
}
```

Також виконала команду **rs.status()** для перевірки стану replica set.
Результат підтвердив успішне налаштування:

```
> rs.status()
< {
  set: 'rs0',
  date: 2026-04-28T15:41:55.983Z,
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
    lastCommittedWallTime: 2026-04-28T15:41:53.332Z,
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
    durableOpTime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
    writtenOpTime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
    lastAppliedWallTime: 2026-04-28T15:41:53.332Z,
    lastDurableWallTime: 2026-04-28T15:41:53.332Z,
    lastWrittenWallTime: 2026-04-28T15:41:53.332Z
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1777390869, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: 2026-04-28T15:40:23.173Z,
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1777390823, i: 1 }), t: Long('-1') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1777390823, i: 1 }), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1777390823, i: 1 }), t: Long('-1') },
    numVotesNeeded: 1,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    newTermStartDate: 2026-04-28T15:40:23.241Z,
    wMajorityWriteAvailabilityDate: 2026-04-28T15:40:23.294Z
  }
}
```

```

    },
    members: [
      {
        _id: 0,
        name: 'localhost:27017',
        health: 1,
        state: 1,
        stateStr: 'PRIMARY',
        uptime: 282,
        optime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeDate: 2026-04-28T15:41:53.000Z,
        optimeWritten: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeWrittenDate: 2026-04-28T15:41:53.000Z,
        lastAppliedWallTime: 2026-04-28T15:41:53.332Z,
        lastDurableWallTime: 2026-04-28T15:41:53.332Z,
        lastWrittenWallTime: 2026-04-28T15:41:53.332Z,
        syncSourceHost: '',
        syncSourceId: -1,
        infoMessage: 'Could not find member to sync from',
        electionTime: Timestamp({ t: 1777390823, i: 2 }),
        electionDate: 2026-04-28T15:40:23.000Z,
        configVersion: 5,
        configTerm: 1,
        self: true,
        lastHeartbeatMessage: ''
      },
      {
        _id: 1,
        name: 'localhost:27018',
        health: 1,
        state: 2,
        stateStr: 'SECONDARY',
        uptime: 50,
        optime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeDurable: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },

```

```

        optime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeDurable: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeWritten: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeDate: 2026-04-28T15:41:53.000Z,
        optimeDurableDate: 2026-04-28T15:41:53.000Z,
        optimeWrittenDate: 2026-04-28T15:41:53.000Z,
        lastAppliedWallTime: 2026-04-28T15:41:53.332Z,
        lastDurableWallTime: 2026-04-28T15:41:53.332Z,
        lastWrittenWallTime: 2026-04-28T15:41:53.332Z,
        lastHeartbeat: 2026-04-28T15:41:54.479Z,
        lastHeartbeatRecv: 2026-04-28T15:41:54.480Z,
        pingMs: Long('0'),
        lastHeartbeatMessage: '',
        syncSourceHost: 'localhost:27017',
        syncSourceId: 0,
        infoMessage: '',
        configVersion: 5,
        configTerm: 1
      },
      {
        _id: 2,
        name: 'localhost:27019',
        health: 1,
        state: 2,
        stateStr: 'SECONDARY',
        uptime: 31,
        optime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeDurable: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeWritten: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
        optimeDate: 2026-04-28T15:41:53.000Z,
        optimeDurableDate: 2026-04-28T15:41:53.000Z,
        optimeWrittenDate: 2026-04-28T15:41:53.000Z,
        lastAppliedWallTime: 2026-04-28T15:41:53.332Z,
        lastDurableWallTime: 2026-04-28T15:41:53.332Z,
        lastWrittenWallTime: 2026-04-28T15:41:53.332Z,
        lastHeartbeat: 2026-04-28T15:41:54.480Z,
        lastHeartbeatRecv: 2026-04-28T15:41:54.975Z,

```

```

{
  _id: 2,
  name: 'localhost:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 31,
  optime: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1777390913, i: 1 }), t: Long('1') },
  optimeDate: 2026-04-28T15:41:53.000Z,
  optimeDurableDate: 2026-04-28T15:41:53.000Z,
  optimeWrittenDate: 2026-04-28T15:41:53.000Z,
  lastAppliedWallTime: 2026-04-28T15:41:53.332Z,
  lastDurableWallTime: 2026-04-28T15:41:53.332Z,
  lastWrittenWallTime: 2026-04-28T15:41:53.332Z,
  lastHeartbeat: 2026-04-28T15:41:54.480Z,
  lastHeartbeatRecv: 2026-04-28T15:41:54.975Z,
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 5,
  configTerm: 1
}
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1777390913, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1777390913, i: 1 })
}

```

Всі три вузли активні (health: 1), votingMembersCount: 3, ok: 1. Secondary вузли синхронізуються з Primary через syncSourceHost: localhost:27017.

У базі **local** є системні колекції реплікації: oplog.rs (лог операцій), replset.election, replset.initialSyncId — що підтверджує активну роботу replica set:

lab06 > local

[Open MongoDB shell](#)
[+ Create collection](#)
[Refresh](#)

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
oplog.rs	CAPPED	36.86 kB	8.24 kB	54	152.00 B	0	0 B
replset.election	-	36.86 kB	60.00 B	1	60.00 B	1	20.48 kB
replset.initialSyncId	-	20.48 kB	49.00 B	1	49.00 B	1	20.48 kB
replset.minvalid	-	20.48 kB	46.00 B	1	46.00 B	1	20.48 kB
replset.oplogTruncateAfterPoint	-	36.86 kB	71.00 B	1	71.00 B	1	20.48 kB
startup_log	CAPPED	20.48 kB	3.56 kB	1	3.56 kB	1	20.48 kB

Щоб перевірити як система поводить ся при відключенні Primary — закрито вікно PowerShell де був запуснений MongoDB на порту **27017**.

Після відключення спроба виконати `rs.status()` через Compass повернула помилку `ECONNREFUSED 127.0.0.1:27017` — підключення відхилено, бо сервер на цьому порту вже не працює:

```
> rs.status()
MongoServerError: connect ECONNREFUSED 127.0.0.1:27017, connect ECONNREFUSED ::1:27017
```

Далі відкрила новий термінал PowerShell і виконала підключення до порту **27018**:

```
PS C:\Users\Admin> mongosh --port 27018
Current Mongosh Log ID: 69f0de49996f5a3ff3628c9f
Connecting to:      mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:      8.2.4
Using Mongosh:       2.6.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-04-28T18:37:36.425+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

rs0 [direct: primary] test>
```

У рядку підключення відобразилось `rs0 [direct: primary]` — це означає що вузол **27018** автоматично став новим **Primary** без будь-якого ручного втручання. MongoDB самостійно провів нові вибори серед **Secondary** вузлів і обрав новий **Primary**.

Далі на **Primary** вузлі (порт 27018) виконала вставку тестового документу в колекцію `test_repl`:

```
rs0 [direct: primary] test> use performance_test
switched to db performance_test
rs0 [direct: primary] performance_test> db.test_repl.insertOne({ msg: "hello from primary", date: new Date() })
{ acknowledged: true, insertedId: ObjectId('69f0de6e996f5a3ff3628ca0') }
rs0 [direct: primary] performance_test>
```

Результат: `acknowledged: true` — документ успішно збережено з `insertedId`.

Потім відкрила новий термінал і підключено до **Secondary** вузла (порт 27019):

```

PS C:\Users\Admin> mongosh --port 27019
Current Mongosh Log ID: 69f0de8c9f117973df628c9f
Connecting to:      mongodb://127.0.0.1:27019/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:      8.2.4
Using Mongosh:       2.6.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-04-28T18:39:01.838+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

rs0 [direct: secondary] test> rs.secondaryOk()
DeprecationWarning: .setSecondaryOk() is deprecated. Use .setReadPref("primaryPreferred") instead
Setting read preference from "primary" to "primaryPreferred"

rs0 [direct: secondary] test> use performance_test
switched to db performance_test
rs0 [direct: secondary] performance_test> db.test_repl.find()
[
  {
    _id: ObjectId('69f0de6e996f5a3ff3628ca0'),
    msg: 'hello from primary',
    date: ISODate('2026-04-28T16:21:02.147Z')
  }
]
rs0 [direct: secondary] performance_test>

```

У результаті на Secondary знайдено той самий документ з ідентичним ObjectId('69f0de6e996f5a3ff3628ca0'), msg: 'hello from primary' та датою 2026-04-28T16:21:02.147Z

Це підтверджує що **реплікація працює коректно** — документ записаний на Primary автоматично з'явився на Secondary вузлі без будь-яких додаткових дій.

Частина 3: Реалізація шардінгу у MongoDB

1. Запустити два екземпляри MongoDB у режимі шардів.

Створила три окремі директорії для зберігання даних config server та двох шардів:

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\Admin> New-Item -ItemType Directory -Path D:\data\cfg -Force

Каталог: D:\data

Mode                LastWriteTime         Length Name
----                -
d-----          28.04.2026   19:32             cfg

PS C:\Users\Admin> New-Item -ItemType Directory -Path D:\data\shard1 -Force

Каталог: D:\data

Mode                LastWriteTime         Length Name
----                -
d-----          28.04.2026   19:32             shard1

PS C:\Users\Admin> New-Item -ItemType Directory -Path D:\data\shard2 -Force
```

```
Windows PowerShell

Mode                LastWriteTime         Length Name
----                -
d-----          28.04.2026   19:32             cfg

PS C:\Users\Admin> New-Item -ItemType Directory -Path D:\data\shard1 -Force

Каталог: D:\data

Mode                LastWriteTime         Length Name
----                -
d-----          28.04.2026   19:32             shard1

PS C:\Users\Admin> New-Item -ItemType Directory -Path D:\data\shard2 -Force

Каталог: D:\data

Mode                LastWriteTime         Length Name
----                -
d-----          28.04.2026   19:32             shard2

PS C:\Users\Admin>
```

Запустила три экземпляры MongoDB у окремих вікнах PowerShell:

Config server (зберігає метадані кластера):

```
Windows PowerShell

PS C:\Users\Admin> mongod --configsvr --replSet cfg --port 27030 --dbpath D:\data\cfg --bind_ip localhost
```

Перший шард:


```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\Admin> mongod --shardsvr --replSet shard1rs --port 27021 --dbpath D:\data\shard1 --bind_ip localhost
```

Другий шард:

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\Admin> mongod --shardsvr --replSet shard2rs --port 27022 --dbpath D:\data\shard2 --bind_ip localhost
```

Підключилась до config server через mongosh:

```
PS C:\Users\Admin> mongosh --port 27030
Current Mongosh Log ID: 69f0e32d534b3130da628c9f
Connecting to: mongod://127.0.0.1:27030/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB: 8.2.4
Using Mongosh: 2.6.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-04-28T19:35:00.492+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----
test>
```

Далі виконала ініціалізацію replica set:

```
test> rs.initiate({
...   _id: "cfg",
...   configsvr: true,
...   members: [{ _id: 0, host: "localhost:27030" }]
... })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777394497, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777394497, i: 1 })
}
cfg [direct: secondary] test>
```

Результат: ok: 1 — config server успішно ініціалізовано. Config server зберігає метадані шардованого кластера (інформацію про розподіл даних між шардами).

Підключилась до першого шарду:

```
PS C:\Users\Admin> mongosh --port 27021
Current Mongosh Log ID: 69f0e3d79ca9f3df1e628c9f
Connecting to:      mongodb://127.0.0.1:27021/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:      8.2.4
Using Mongosh:       2.6.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-04-28T19:35:17.869+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test>
```

Ініціалізувала replica set:

```
test> rs.initiate({ _id: "shard1rs", members: [{ _id: 0, host: "localhost:27021" }] })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777394672, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777394672, i: 1 })
}
shard1rs [direct: secondary] test>
```

Результат: ok: 1 — перший шард успішно ініціалізовано.

Підключилась до другого шарду:

```
PS C:\Users\Admin> mongosh --port 27022
Current Mongosh Log ID: 69f0e41ddb99ffdbd628c9f
Connecting to:      mongodb://127.0.0.1:27022/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:      8.2.4
Using Mongosh:       2.6.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-04-28T19:35:35.776+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test>
```

Ініціалізувала replica set:

```
test> rs.initiate({ _id: "shard2rs", members: [{ _id: 0, host: "localhost:27022" }] })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777394750, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA= ', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777394750, i: 1 })
}
shard2rs [direct: secondary] test>
```

Результат: ok: 1 — другий шард успішно ініціалізовано.

Запустила mongos — маршрутизатор запитів шардованого кластера. Саме до нього підключаються клієнти, а він розподіляє запити між шардами. У параметрі --configdb вказано ім'я config server replica set та його адресу:\

```
PS C:\Users\Admin> mongos --configdb cfg/localhost:27030 --port 27020 --bind_ip localhost
{"t":{"$date":"2026-04-28T19:47:29.742+03:00"},"s":"I", "c":"-", "id":8991200, "ctx":"thread1","msg":"Shuffling
initializers","attr":{"seed":42487871}}
{"t":{"$date":"2026-04-28T19:47:29.767+03:00"},"s":"W", "c":"SHARDING", "id":24132, "ctx":"thread1","msg":"Running a
sharded cluster with fewer than 3 config servers should only be done for testing purposes and is not recommended for pro
duction."}
```

Підключилась до маршрутизатора mongos:

```
PS C:\Users\Admin> mongosh --port 27020
Current Mongosh Log ID: 69f0e4eb5187a24d20628c9f
Connecting to:      mongodb://127.0.0.1:27020/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.6.0
Using MongoDB:      8.2.4
Using Mongosh:       2.6.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-04-28T19:47:29.866+03:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
-----

[direct: mongos] test>
```

2. Додати шарди через MongoDB Shell.

Додала перший та другий шарди у кластер:

```
[direct: mongos] test> sh.addShard("shard1rs/localhost:27021")
{
  shardAdded: 'shard1rs',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777394941, i: 58 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777394941, i: 58 })
}
[direct: mongos] test> sh.addShard("shard2rs/localhost:27022")
{
  shardAdded: 'shard2rs',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777394948, i: 50 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777394948, i: 50 })
}
[direct: mongos] test>
```

Увімкнула шардінг для бази даних **performance_test**:

```
[direct: mongos] test> sh.enableSharding("performance_test")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777395024, i: 21 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777395024, i: 21 })
}
[direct: mongos] test>
```

Увімкнула шардінг для колекції **sales** з ключем шардування **category**.
Ключ шардування визначає як документи розподіляються між шардами:

```
[direct: mongos] test> sh.shardCollection("performance_test.sales", { category: 1 })
{
  collectionssharded: 'performance_test.sales',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777395065, i: 32 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1777395065, i: 31 })
}
[direct: mongos] test>
```

3. Перевірити розподіл даних та оцінити ефективність масштабування.

Перевірила конфігурацію шардованого кластера:

```
[direct: mongos] test> sh.status()
shardingVersion
{
  _id: ObjectId('69f0e342485aa74f7659cae3'),
  clusterId: ObjectId('69f0e342485aa74f7659cae2')
}
---
shards
[
  {
    _id: 'shard1rs',
    host: 'shard1rs/localhost:27021',
    state: 1,
    topologyTime: Timestamp({ t: 1777394941, i: 32 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'shard2rs',
    host: 'shard2rs/localhost:27022',
    state: 1,
    topologyTime: Timestamp({ t: 1777394948, i: 27 }),
    replSetConfigVersion: Long('-1')
  }
]
---
active mongoses
[ { '8.2.4': 1 } ]
---
autosplit
```

```

{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[
  {
    ns: 'config.system.sessions',
    shards: [
      {
        shardName: 'shard1rs',
        numOrphanedDocs: 0,
        numOwnedDocuments: 11,
        ownedSizeBytes: 1089,
        orphanedSizeBytes: 0
      }
    ]
  },
  {
    ns: 'performance_test.sales',
    shards: [
      {
        shardName: 'shard2rs',
        numOrphanedDocs: 0,

```

```

        numOwnedDocuments: 0,
        ownedSizeBytes: 0,
        orphanedSizeBytes: 0
      }
    ]
  }
]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {
      'config.system.sessions': {
        shardKey: { _id: 1 },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [ { shard: 'shard1rs', nChunks: 1 } ],
        chunks: [
          { min: { _id: MinKey() }, max: { _id: MaxKey() }, 'on shard': 'shard1rs', 'last modified': Timestamp({ t: 1, i
: 0 }) }
        ],
        tags: []
      }
    }
  },
  {
    database: {
      _id: 'performance_test',

```

```

    }
  },
  {
    database: {
      _id: 'performance_test',
      primary: 'shard2rs',
      version: {
        uuid: UUID('e15b26c5-0767-4edd-8517-3de0c040feb4'),
        timestamp: Timestamp({ t: 1777395024, i: 9 }),
        lastMod: 1
      }
    },
    collections: {
      'performance_test.sales': {
        shardKey: { category: 1 },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [ { shard: 'shard2rs', nChunks: 1 } ],
        chunks: [
          { min: { category: MinKey() }, max: { category: MaxKey() }, 'on shard': 'shard2rs', 'last modified': Timestamp
            ({ t: 1, i: 0 }) }
        ],
        tags: []
      }
    }
  }
]
[direct: mongos] test>

```

Команда **sh.status()** підтвердила що обидва шарди активні, балансувальник увімкнений і колекція розподілена між шардами. Шардинг дозволяє горизонтально масштабувати базу даних при зростанні обсягу даних.